

Documentación

EasyVMeter

José Miguel Allés Morales
28/04/2026

Índice

Introducción: Rompiendo la barrera del sonido visual.....	3
Hardware necesario.....	4
Instalación del software en la RP2040.....	4
Configuración del DAW.....	4
■ Ableton Live:.....	4
■ FL studio:.....	5
■ Cubase:.....	5
■ Logic Pro:.....	5
■ Studio One:.....	5
■ BitWig:.....	5
■ Reaper:.....	5
■ CakeWalk:.....	5
Código comentado.....	6
boot.py.....	6
code.py.....	8
Resumen de funcionamiento del código.....	18
1. Configuración de Hardware.....	18
2. Los Dos Modos de Funcionamiento.....	18
A. Modo Control MIDI (Predeterminado).....	18
B. Modo Solo.....	18
3. Funciones Especiales y Seguridad.....	19
4. Flujo del Programa (Bucle Principal).....	19
Esquema de conexionado.....	20
Documentación:.....	21

Introducción: Rompiendo la barrera del sonido visual

Easy Vmeter nace de una necesidad real en el aula de sonido: la autonomía. Durante años de trabajo con alumnos invidentes utilizando Reaper y OSARA, identificamos un muro invisible pero crítico: la gestión de la ganancia de entrada. Mientras que los productores videntes cuentan con vúmetros visuales constantes, las herramientas de accesibilidad actuales suelen avisar cuando el error ya ha ocurrido: el clip.

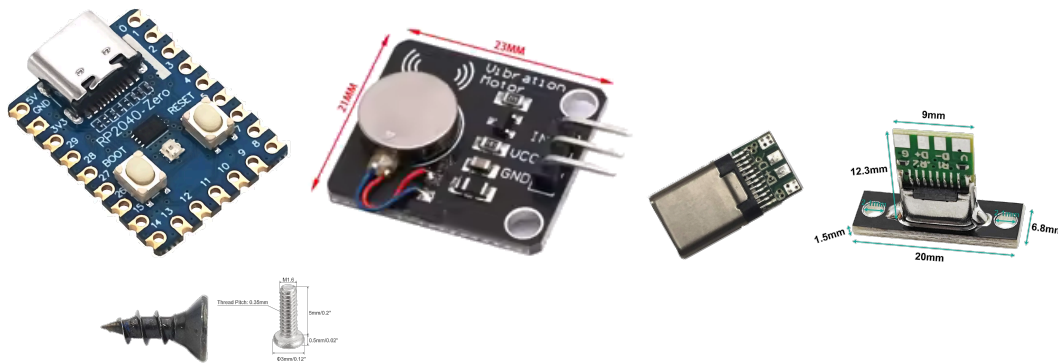
Este proyecto transforma esa limitación en una solución tangible. Easy Vmeter es un dispositivo de código abierto que traduce los niveles de señal en vibraciones físicas, permitiendo que el tacto sustituya a la vista en una de las fases más importantes de la grabación. Al emular una superficie de control MIDI bajo el protocolo estándar Mackie Control, ofrecemos una herramienta universal, precisa y, sobre todo, integradora para cualquier flujo de trabajo profesional.

Especificaciones Clave

- **Procesador:** RP2040.
- **Protocolo:** MIDI / Mackie Control.
- **Salida:** Vibración PWM e indicador LED NeoPixel.
- **Licencia:** Open Source.

Hardware necesario

- RP2040-Zero RP2040
- Interruptor de Motor de vibración PWM
- USB Tipo C 3.1 Macho para Soldar
- USB 3.1 Tipo C Hembra, Conector De Soldadura de Montaje en Panel
- Tornillos: 4x Cabeza Redonda Plana M1,6-0,35x5mm (Carcasa mini)
- Tornillos: 2x Cabeza Redonda Plana M1,6-0,35x5mm + 2x Cabeza Avellanada Plana 2,5 x 10mm (Carcasa para pulsera)



Instalación del software en la RP2040

- Después de conectar la unidad a través del puerto usb, se abrirá la unidad de almacenamiento del dispositivo. En caso de no abrirse, simplemente buscarlo en el equipo.
- Copiar el archivo del firmware
“adafruit-circuitpython-waveshare_rp2040_zero-es-9.2.7.uf2” en la unidad. El dispositivo se reiniciará automáticamente, en caso contrario, reiniciar de manera manual.
- Copiar todos los archivos y carpetas de EasyVMeter para RP2040 - Zero. Éstos son los dos archivos para configuración y programación ([boot.py](#) y [code.py](#)) y las librerías necesarias (adafruit midi y neopixel)
- Reiniciar el dispositivo.

Configuración del DAW

■ Ableton Live:

Abra el menú de preferencias en Ableton Live, establezca la superficie de control en "MackieControl" y seleccione "EasyVMeter" tanto para la entrada como para la salida.

■ FL studio:

Acceda a la configuración MIDI en FL Studio, habilite "EasyVMeter", configure el tipo de controlador en "Mackie Control Universal" y asegúrese de que la entrada y salida del SMC-Mixer estén en el mismo puerto.

■ Cubase:

En Cubase, vaya a Configuración de estudio, agregue el dispositivo "Mackie Control" y elija "EasyVMeter" tanto para la entrada como para la salida MIDI.

■ Logic Pro:

En Logic Pro, vaya a Superficies de control > Nueva > Instalar, agregue "Mackie Control" y establezca tanto el puerto de salida como el puerto de entrada en "EasyVMeter".

■ Studio One:

En Studio One, visite Opciones > Dispositivos externos, agregue "Mackie Control" y configure tanto "Recibir desde" como "Enviar a" en "EasyVMeter".

■ BitWig:

En Configuración > Controladores de BitWig, agregue el controlador "Mackie Control" y designe "EasyVMeter" tanto para la entrada como para la salida MIDI.

■ Reaper:

En Reaper, vaya a Preferencias > Control/OSC/web, agregue "Mackie Control Universal" y seleccione "EasyVMeter" tanto para la entrada como para la salida MIDI.

■ CakeWalk:

En CakeWalk, ingrese a Preferencias > Superficies de control, incorpore "Mackie Control" y elija "EasyVMeter" tanto para la entrada como para la salida.

Código comentado

boot.py

```
import usb_midi # Importa la librería para configurar el dispositivo USB como MIDI.
import storage   # Importa la librería para controlar el almacenamiento USB.

# --- Configuración de Nombres del Dispositivo MIDI USB ---
# Este bloque 'try...except' intenta establecer nombres descriptivos para los interfaces MIDI
# USB.
# Esto ayuda a identificar el dispositivo más fácilmente en DAWs (Digital Audio
# Workstations)
# y otros programas que listan dispositivos MIDI.
try:
    # usb_midi.set_names() permite personalizar los nombres que aparecen en el sistema
    # operativo.
    # 'streaming_interface_name': Nombre del interfaz de flujo principal (a menudo la "tarjeta
    # de sonido" virtual).
    # 'audio_control_interface_name': Nombre del interfaz de control de audio.
    # 'in_jack_name': Nombre del puerto MIDI de entrada visible en el software.
    # 'out_jack_name': Nombre del puerto MIDI de salida visible en el software.
    usb_midi.set_names(
        streaming_interface_name="strEasyVMeter",
        audio_control_interface_name="EasyVMeter",
        in_jack_name="EasyVMeter In",
        out_jack_name="EasyVMeter Out"
    )
    # Mensaje de confirmación en la consola serial si la operación fue exitosa.
    print("Nombres MIDI establecidos en boot.py usando usb_midi.set_names")
except AttributeError:
    # Si la función usb_midi.set_names no existe en esta versión de CircuitPython,
    # o si los parámetros son incorrectos, se capturará un AttributeError.
    # Esto es una advertencia, ya que el dispositivo seguirá funcionando, pero con nombres
    # por defecto.
    print("Advertencia: No se pudo usar usb_midi.set_names. Es posible que esta función no
    exista en la versión de CircuitPython.")

# --- Desactivación del Acceso al Disco USB ---
# Este bloque 'try...except' intenta deshabilitar el acceso al almacenamiento USB (como una
# unidad de disco).
# Esto es útil en dispositivos que no necesitan ser vistos como un disco para el usuario final,
# liberando recursos y evitando problemas de corrupción si el usuario intenta escribir en él.
try:
    # storage.disable_usb_drive() desactiva la función de unidad de disco USB del
    # dispositivo.
    storage.disable_usb_drive()
    # Mensaje de confirmación en la consola serial si la operación fue exitosa.
```

```
    print("Usb desactivado")
except AttributeError:
    # Si la función storage.disable_usb_drive no existe en esta versión de CircuitPython,
    # se capturará un AttributeError. El dispositivo podría seguir mostrando el disco USB.
    print("Advertencia: No se pudo usar storage.disable_usb_drive. Es posible que esta
función no exista en la versión de CircuitPython.")
```

code.py

```
# Importación de librerías necesarias para el proyecto CircuitPython
import usb_midi    # Permite la comunicación MIDI a través de USB
import adafruit_midi # Proporciona una interfaz de alto nivel para enviar y recibir mensajes MIDI
import board       # Acceso a los pines físicos de la placa CircuitPython
import time        # Funciones para manejar el tiempo, como delays (sleep) y medición de tiempo (monotonic)
import pwmio       # Permite generar señales PWM (Pulse Width Modulation) para controlar el vibrador
import neopixel    # Control de LEDs NeoPixel (como el LED de estado integrado en la placa)

# Importaciones específicas de tipos de mensajes MIDI para mayor claridad y eficiencia
from adafruit_midi.channel_pressure import ChannelPressure # Para mensajes de presión de canal (medidor de volumen/clip)
from adafruit_midi.note_on import NoteOn                 # Para mensajes de nota encendida (usados para cambio de modo)

# --- Constantes de Configuración de Hardware ---

# Pin GPIO de la placa Raspberry Pi RP2040-Zero conectado al vibrador para la salida PWM
PWM_PIN = board.GP29
# Frecuencia de la señal PWM en Hertz (Hz). 150 Hz es una frecuencia común para motores de vibración.
PWM_FREQUENCY = 150
# Valor máximo del ciclo de trabajo PWM (65535 representa el 100% de encendido).
PWM_DUTY_CYCLE_MAX = 65535
# Valor mínimo del ciclo de trabajo PWM (0 representa el 0% de encendido, es decir, apagado).
PWM_DUTY_CYCLE_MIN = 0

# Duración del período de "enfriamiento" (cooldown) en segundos para la animación de clip.
# Una vez que la animación de clip termina, no se puede volver a activar hasta que pase este tiempo.
CLIP_ANIMATION_COOLDOWN_SECONDS = 1.0
# Variable que almacena el momento en el que la animación de clip estará lista para ser activada de nuevo.
# Se inicializa en 0.0 para permitir que la primera animación se active de inmediato al inicio.
clip_animation_ready_time = 0.0

# Pin del NeoPixel (LED de estado) en la placa.
# 'getattr(board, 'NEOPIXEL', board.GP16)' intenta usar el pin estándar 'board.NEOPIXEL'.
# Si 'board.NEOPIXEL' no está definido para esta placa, usa 'board.GP16' como alternativa.
NEOPIXEL_PIN = getattr(board, 'NEOPIXEL', board.GP16)
```



```

# Definición de colores RGB (Rojo, Verde, Azul) para el NeoPixel.
# Cada componente de color va de 0 a 255.
COLOR_OK = (0, 255, 0)      # Verde: Indica que el sistema está listo y funcionando
correctamente.
COLOR_ERROR = (255, 0, 0)    # Rojo: Indica un error o un evento de clip.
COLOR_RECIVE = (255, 255, 0) # Amarillo: Indica que se están recibiendo mensajes
MIDI de vibración activa.
COLOR_SOLO = (0, 0, 255)     # Azul: Indicador visual para el "Modo Solo".
COLOR_WHITE = (255, 255, 255) # Blanco: Color general, usado en la animación de
inicio.
COLOR_OFF = (0, 0, 0)       # Negro: Para apagar completamente el LED.

# Constante para el número de nota MIDI que se usará para cambiar de modo.
SOLO_TRIGGER_NOTE = 08

# Inicialización del objeto NeoPixel.
# Se conecta al pin definido, controla 1 píxel, con brillo máximo (1) y no se actualiza
automáticamente.
# 'auto_write=False' significa que los cambios de color solo se aplican cuando se llama a
'pixels.show()'.
pixels = neopixel.NeoPixel(NEOPIXEL_PIN, 1, brightness=1, auto_write=False)

# Variable global para el objeto PWM que controlará el vibrador.
# Se inicializa a None y se asignará en el bloque de inicialización principal.
pwm = None

# --- Funciones de Control del LED de Estado ---

def led_error():
    """Establece el LED de estado en color rojo para indicar un error o un evento de clip."""
    pixels.fill(COLOR_ERROR) # Rellena el NeoPixel con el color rojo
    pixels.show()            # Muestra el color en el LED

def led_ok():
    """Establece el LED de estado en color verde para indicar que el sistema está listo y
operativo."""
    pixels.fill(COLOR_OK)    # Rellena el NeoPixel con el color verde
    pixels.show()            # Muestra el color en el LED

def led_recive():
    """Establece el LED de estado en color amarillo para indicar que se están recibiendo
mensajes MIDI activos."""
    pixels.fill(COLOR_RECIVE)# Rellena el NeoPixel con el color amarillo
    pixels.show()            # Muestra el color en el LED

def led_off():
    """Apaga completamente el LED de estado."""

```

```
pixels.fill(COLOR_OFF) # Rellena el NeoPixel con el color negro (apagado)
pixels.show()          # Muestra el color en el LED
```

```
def led_solo():
```

```
    """Establece el LED de estado en color azul, indicando que el dispositivo está en 'Modo
Solo'."""
```

```
    pixels.fill(COLOR_SOLO) # Rellena el NeoPixel con el color azul
    pixels.show()          # Muestra el color en el LED
```

```
# --- Funciones de Animación del Vibrador ---
```

```
def startup_animation():
```

```
    """
```

```
    Ejecuta una animación de inicio en el vibrador y el LED.
```

```
    Consiste en una vibración a máxima intensidad seguida de un desvanecimiento gradual.
```

```
    """
```

```
    if pwm: # Verifica que el objeto PWM se haya inicializado correctamente
```

```
        # Fase 1: Vibrar a máxima intensidad y encender LED verde
```

```
        pwm.duty_cycle = PWM_DUTY_CYCLE_MAX # Establece el vibrador al 100% de
potencia
```

```
        pixels.fill(COLOR_OK) # Pone el LED en verde durante la vibración inicial
```

```
        pixels.show()
```

```
        time.sleep(1) # Mantiene la vibración y el LED encendidos por 1 segundo
```

```
        # Fase 2: Desvanecimiento gradual (fade-out) del vibrador
```

```
        fade_steps = 100 # Número de pasos para el desvanecimiento (más pasos = más
suave)
```

```
        fade_delay = 2.0 / fade_steps # Retardo entre cada paso para que el desvanecimiento
dure 2 segundos
```

```
        # Calcula cuánto debe disminuir el ciclo de trabajo en cada paso del desvanecimiento
```

```
        duty_cycle_decrease_per_step = (PWM_DUTY_CYCLE_MAX -
PWM_DUTY_CYCLE_MIN) / (fade_steps - 1) if fade_steps > 1 else 0
```

```
        # Itera a través de los pasos para reducir el ciclo de trabajo del vibrador
```

```
        for i in range(fade_steps):
```

```
            # Calcula el ciclo de trabajo actual para el paso 'i'
```

```
            duty_cycle = PWM_DUTY_CYCLE_MAX - int(duty_cycle_decrease_per_step * i)
```

```
            # Asegura que el ciclo de trabajo no caiga por debajo del mínimo (0)
```

```
            duty_cycle = max(PWM_DUTY_CYCLE_MIN, duty_cycle)
```

```
        pwm.duty_cycle = duty_cycle # Aplica el nuevo ciclo de trabajo al vibrador
```

```
        time.sleep(fade_delay) # Espera un breve momento antes del siguiente paso
```

```
        # Asegura que el vibrador esté completamente apagado y el LED en verde al final de la
animación
```

```
        pwm.duty_cycle = PWM_DUTY_CYCLE_MIN
```

```
        pixels.fill(COLOR_OK)
```

```

pixels.show()

# Bandera global para controlar si la animación de clip está en curso.
# Esto evita que se ejecuten múltiples animaciones de clip simultáneamente.
clipping_is_playing = False

def trigger_clip_animation():
    """
    Ejecuta una animación en el vibrador y el LED de estado para indicar un evento de clip.
    Implementa un mecanismo de "cooldown" para evitar que la animación se dispare
    repetidamente.
    """
    global clipping_is_playing
    global last_message_time
    global clip_animation_ready_time # Variable para controlar el cooldown

    current_time = time.monotonic() # Obtiene el tiempo actual para las comprobaciones de
    cooldown

    # --- Lógica de "Gate" para el Cooldown ---
    # Si la animación ya está en curso (clipping_is_playing es True),
    # O si el tiempo actual es menor que el tiempo en el que la animación estará lista (todavía
    en cooldown),
    # entonces ignora este trigger y sale de la función.
    if clipping_is_playing or (current_time < clip_animation_ready_time):
        return

    # Si llegamos a este punto, significa que la animación NO está en curso y el cooldown ha
    terminado,
    # por lo tanto, podemos iniciar una nueva animación de clip.
    last_message_time = current_time # Actualiza el tiempo del último mensaje relevante
    para el timeout principal
    clipping_is_playing = True # Marca que la animación de clip está en curso

    if pwm: # Verifica que el objeto PWM se haya inicializado correctamente
        # Lógica de la animación de clip: el vibrador y el LED parpadean en un patrón
        específico.
        for _ in range(2): # Repetir 2 veces el patrón principal de parpadeo
            pwm.duty_cycle = PWM_DUTY_CYCLE_MAX # Vibrador al 100% (ON)
            led_error() # LED rojo (indicando clip)
            time.sleep(0.2) # Vibrador y LED encendidos por 0.2 segundos

            pwm.duty_cycle = PWM_DUTY_CYCLE_MIN # Vibrador apagado (OFF)
            led_recive() # LED amarillo (indicando que está recibiendo/listo)
            time.sleep(0.2) # Vibrador y LED apagados/amarillos por 0.2 segundos

            pwm.duty_cycle = PWM_DUTY_CYCLE_MAX # Vibrador al 100% (ON)
            led_error() # LED rojo

```

```

time.sleep(0.2)

pwm.duty_cycle = PWM_DUTY_CYCLE_MIN # Vibrador apagado (OFF)
led_recive() # LED amarillo
time.sleep(0.2)

pwm.duty_cycle = PWM_DUTY_CYCLE_MAX # Vibrador al 100% (ON)
led_error() # LED rojo
time.sleep(0.2)

pwm.duty_cycle = PWM_DUTY_CYCLE_MIN # Vibrador apagado (OFF)
led_recive() # LED amarillo
time.sleep(0.4) # Pausa más larga al final de este ciclo para un efecto distintivo

# Asegura que el vibrador termine completamente apagado al finalizar la animación
pwm.duty_cycle = PWM_DUTY_CYCLE_MIN

clipping_is_playing = False # Marca que la animación de clip ha terminado
last_message_time = time.monotonic() # Reinicia el timeout principal después de que la
animación finaliza

# Establece el tiempo en el que la próxima animación de clip será permitida.
# Esto inicia el período de cooldown, asegurando que no se dispare de nuevo
inmediatamente.
clip_animation_ready_time = time.monotonic() +
CLIP_ANIMATION_COOLDOWN_SECONDS

# --- Bloque de Inicialización de Hardware y MIDI ---

# Intenta inicializar el objeto PWM para controlar el vibrador.
try:
    pwm = pwmio.PWMOut(PWM_PIN, frequency=PWM_FREQUENCY,
duty_cycle=PWM_DUTY_CYCLE_MIN)
except Exception as e:
    # Si hay un error crítico al configurar el PWM, el LED se pone rojo y el script se detiene
    # en un bucle infinito para indicar un fallo irrecuperable.
    pixels.fill(COLOR_ERROR)
    pixels.show()
    while True:
        pass

# Variables para la configuración MIDI
midi = None # Objeto MIDI, se inicializará si se encuentra un puerto MIDI de entrada.
midi_setup_successful = False # Bandera para indicar si la inicialización MIDI fue exitosa.

# Busca un puerto MIDI de entrada USB disponible en la placa.
midi_in_port = None
for port in usb_midi.ports:

```

```

if isinstance(port, usb_midi.PortIn): # Comprueba si el puerto es de entrada
    midi_in_port = port # Asigna el puerto encontrado
    break # Sale del bucle una vez que encuentra el primer puerto de entrada.

# Intenta inicializar la librería adafruit_midi si se encontró un puerto MIDI de entrada.
if midi_in_port:
    try:
        midi = adafruit_midi.MIDI(midi_in=midi_in_port) # Inicializa el objeto MIDI
        midi_setup_successful = True # Marca la inicialización MIDI como exitosa
        led_ok() # Enciende el LED en verde para indicar que el sistema está listo.
        startup_animation() # Ejecuta la animación de inicio del vibrador.

    except Exception as e:
        # Si hay un error al inicializar adafruit_midi, marca el fallo y enciende el LED rojo.
        midi_setup_successful = False
        led_error()

else:
    # Si no se encontró ningún puerto MIDI de entrada USB, marca el fallo y enciende el LED
    rojo.
    midi_setup_successful = False
    led_error()

# --- Constantes y Variable para la Gestión de Modos de Funcionamiento ---

# Definición de los diferentes modos de funcionamiento del dispositivo mediante constantes
numéricas.
MODE_MIDI_CONTROL = 0 # Modo predeterminado: controla el vibrador con mapeo
normal de volumen.
MODE_SOLO = 1 # Modo "Solo": vibración a partir de un nivel de volumen específico,
mapeo equitativo.
MODE_CONFIG = 2 # Ejemplo de otro modo futuro (por ejemplo, para cambiar
configuraciones).

# Variable global que almacena el modo de funcionamiento actual del dispositivo.
# El dispositivo se inicia por defecto en el "Modo de Control MIDI".
current_mode = MODE_MIDI_CONTROL

# --- Variables de Timeout (Globales, usadas por los modos) ---
# Tiempo en segundos que debe pasar sin mensajes MIDI "activos" antes de que el vibrador
se apague.
TIMEOUT_SECONDS = 0.5
# Registra el tiempo del último mensaje MIDI "activo" recibido.
# Un mensaje "activo" es aquel que debería generar una vibración o un cambio de modo.
last_message_time = time.monotonic()

# --- Funciones para cada Modo de Funcionamiento ---

```

```

def run_midi_control_mode(msg=None):
    """
    Gestiona el funcionamiento del dispositivo en el 'Modo de Control MIDI'.
    Procesa mensajes ChannelPressure para el volumen y el clip, con un mapeo de volumen
    normal (0-11).
    """

    global last_message_time # Permite modificar la variable global 'last_message_time'

    # Manejo del Timeout: Apaga el vibrador si no se reciben mensajes MIDI activos
    # y si no se está reproduciendo la animación de clip.
    if not clipping_is_playing and (time.monotonic() - last_message_time) >
    TIMEOUT_SECONDS:
        if pwm and pwm.duty_cycle != PWM_DUTY_CYCLE_MIN:
            pwm.duty_cycle = PWM_DUTY_CYCLE_MIN # Apaga el vibrador
            led_ok() # El LED se pone verde (OK) si el vibrador se apaga por timeout

    # Procesamiento del mensaje MIDI si se ha recibido uno en esta iteración del bucle
    principal.
    if msg is not None:
        # Si el mensaje es de tipo ChannelPressure (usado para el medidor de volumen/clip)
        if isinstance(msg, ChannelPressure):
            # Y si es del Canal MIDI 0 (el canal específico que estamos monitorizando)
            if msg.channel == 0:
                data_byte = msg.pressure # Obtiene el valor del byte de presión (0-127)
                # Extrae el índice del track (los 4 bits superiores del byte de presión)
                track_index = (data_byte >> 4) & 0x0F
                # Extrae el estado de volumen/clip (los 4 bits inferiores del byte de presión)
                volume_clip_status = data_byte & 0x0F

                # Si el mensaje es para el Track 0 (el track específico que estamos controlando)
                if track_index == 0:
                    # --- Actualización Condicional de last_message_time ---
                    # Solo actualiza el timeout si el mensaje de presión indica actividad (presión >
                    0).

                    # Esto evita que mensajes de presión 0 reinicien el timeout innecesariamente.
                    if volume_clip_status > 0:
                        last_message_time = time.monotonic()

                    # --- Lógica de Control de Volumen (niveles 0 a 11) ---
                    # Si el estado es un nivel de volumen normal (0 a 11)
                    if volume_clip_status <= 0x0B:
                        # Si no estamos en medio de una animación de clip (para evitar
                        interferencias)
                        if not clipping_is_playing:
                            level_0_11 = volume_clip_status # El nivel de volumen real (0 a 11)
                            if pwm: # Verifica que el objeto PWM esté disponible
                                # Si el nivel es 0, apaga el vibrador explícitamente y el LED se pone
                                verde.

```

```

        if level_0_11 == 0:
            pwm.duty_cycle = PWM_DUTY_CYCLE_MIN
            led_ok() # LED verde (indicando que está listo, pero sin vibración
activa)

        else: # Para niveles de 1 a 11, activa la vibración.
            # Mapea el nivel de volumen (1-11) al ciclo de trabajo del vibrador
(0%-100%).

            duty_cycle = int((level_0_11 / 11.0) * PWM_DUTY_CYCLE_MAX)
            # Asegura un mínimo de vibración perceptible incluso para niveles
bajos (por encima de 0).
            duty_cycle = max(PWM_DUTY_CYCLE_MIN + 1000, duty_cycle)
            pwm.duty_cycle = duty_cycle # Aplica el ciclo de trabajo al vibrador
            led_recive() # LED amarillo (indicando vibración activa)

# --- Lógica de Detección de Clip (niveles 12 o 13) ---
# Si el estado es un mensaje de Clip (valores 12 o 13)
elif volume_clip_status == 0x0C or volume_clip_status == 0x0D:
    trigger_clip_animation() # Llama a la función de animación de clip

def run_solo_mode(msg=None):
    """
    Gestiona el funcionamiento del dispositivo en el 'Modo Solo'.
    El vibrador se activa solo a partir del nivel 6 de ChannelPressure.
    Los niveles 0-5 no producen vibración. Los niveles 6-11 se mapean equitativamente.
    """

    global last_message_time # Permite modificar la variable global 'last_message_time'

    # Manejo del Timeout: Apaga el vibrador si no se reciben mensajes MIDI activos
    # y si no se está reproduciendo la animación de clip.
    if not clipping_is_playing and (time.monotonic() - last_message_time) >
TIMEOUT_SECONDS:
        if pwm and pwm.duty_cycle != PWM_DUTY_CYCLE_MIN:
            pwm.duty_cycle = PWM_DUTY_CYCLE_MIN # Apaga el vibrador
            led_solo() # El LED se pone azul (modo SOLO inactivo) si el vibrador se apaga por
timeout

    # Procesamiento del mensaje MIDI si se ha recibido uno en esta iteración del bucle
principal.
    if msg is not None:
        # Si el mensaje es de tipo ChannelPressure
        if isinstance(msg, ChannelPressure):
            # Y si es del Canal MIDI 0
            if msg.channel == 0:
                data_byte = msg.pressure
                track_index = (data_byte >> 4) & 0x0F
                volume_clip_status = data_byte & 0x0F

            # Si el mensaje es para el Track 0

```

```

if track_index == 0:
    # --- Actualización Condicional de last_message_time ---
    # Solo actualiza el timeout si el mensaje de presión está en el rango activo (6 o
más).

    # Esto ignora los mensajes de presión 0-5 para el timeout.
    if volume_clip_status >= 6:
        last_message_time = time.monotonic()

    # --- Lógica de Control de Volumen (niveles 0 a 11) ---
    # Si el estado es un nivel de volumen normal (0 a 11)
    if volume_clip_status <= 0x0B:
        # Si no estamos en medio de una animación de clip
        if not clipping_is_playing:
            # Niveles 0 a 5: No producen vibración.
            if volume_clip_status < 6:
                if pwm:
                    pwm.duty_cycle = PWM_DUTY_CYCLE_MIN # Vibrador apagado
                    led_solo() # LED azul (indicando modo SOLO inactivo)
            # Niveles 6 a 11: Producen vibración, mapeados equitativamente.
            else:
                # Mapea los niveles 6-11 a un rango de 0-5 para la escala de PWM.
                # Hay 6 niveles activos (6,7,8,9,10,11), que se mapean a 5 intervalos
(0-5).

                level_mapped_0_5 = volume_clip_status - 6
                if pwm:
                    # Calcula el ciclo de trabajo: 0 para level_mapped_0_5=0 (MIDI 6),
                    # y MAX para level_mapped_0_5=5 (MIDI 11).
                    duty_cycle = int((level_mapped_0_5 / 5.0) *
PWM_DUTY_CYCLE_MAX)

                    # Asegura un mínimo de vibración para el primer nivel activo (MIDI 6).
                    if duty_cycle > PWM_DUTY_CYCLE_MIN and duty_cycle <
PWM_DUTY_CYCLE_MIN + 1000:
                        duty_cycle = PWM_DUTY_CYCLE_MIN + 1000

                    pwm.duty_cycle = duty_cycle # Aplica el ciclo de trabajo al vibrador
                    led_recive() # LED amarillo (indicando vibración activa)

    # --- Lógica de Detección de Clip (niveles 12 o 13) ---
    # Si el estado es un mensaje de Clip (valores 12 o 13)
    elif volume_clip_status == 0x0C or volume_clip_status == 0x0D:
        trigger_clip_animation() # Llama a la función de animación de clip

# --- Bucle Principal del Programa ---

# El bucle principal se ejecuta indefinidamente, gestionando la recepción MIDI y los
diferentes modos.
while True:

```



```

received_msg = None # Variable para almacenar el mensaje MIDI recibido en esta
iteración.

# Solo intenta recibir mensajes MIDI si la configuración inicial fue exitosa.
if midi_setup_successful:
    received_msg = midi.receive() # Intenta recibir un mensaje MIDI (es no bloqueante).

# Si se recibió un mensaje MIDI en esta iteración
if received_msg is not None:
    # --- Lógica de Cambio de Modo (siempre se comprueba, sin importar el modo
actual) ---
    # Si el mensaje recibido es una NoteOn (potencial comando de cambio de modo)
    if isinstance(received_msg, NoteOn):
        # Y si la nota es la predefinida para el cambio de modo (F#6)
        if received_msg.note == SOLO_TRIGGER_NOTE:
            # Este mensaje es una interacción directa del usuario, así que reinicia el
timeout general.
            last_message_time = time.monotonic()

            # Si la velocidad es 1, cambia a Modo SOLO.
            if received_msg.velocity == 1:
                if current_mode != MODE_SOLO: # Solo cambia si no está ya en este modo
                    current_mode = MODE_SOLO
                    # Apaga el vibrador y establece el LED a azul al cambiar de modo
                    if pwm: pwm.duty_cycle = PWM_DUTY_CYCLE_MIN
                    led_solo()
                    received_msg = None # Consume el mensaje para que no sea procesado
por el modo

                # Si la velocidad es 0 (análogo a Note Off), cambia a Modo de Control MIDI.
                elif received_msg.velocity == 0:
                    if current_mode != MODE_MIDI_CONTROL: # Solo cambia si no está ya en
este modo
                        current_mode = MODE_MIDI_CONTROL
                        # Apaga el vibrador y establece el LED a verde al cambiar de modo
                        if pwm: pwm.duty_cycle = PWM_DUTY_CYCLE_MIN
                        led_ok()
                        received_msg = None # Consume el mensaje

            # IMPORTANTE: Si el mensaje recibido no es un NoteOn para cambio de modo,
            # ni un ChannelPressure en canal 0 (procesado dentro de los modos),
            # 'last_message_time' NO se actualizará. Esto permite que el timeout apague el
vibrador
            # si solo hay "ruido" MIDI irrelevante.

# --- Ejecutar la Lógica del Modo Actual ---
# Si la inicialización MIDI fue exitosa, ejecuta la función correspondiente al modo actual.
if midi_setup_successful:

```

```

    if current_mode == MODE_MIDI_CONTROL:
        run_midi_control_mode(received_msg) # Pasa el mensaje recibido al modo
MIDI_CONTROL
    elif current_mode == MODE_SOLO:
        run_solo_mode(received_msg) # Pasa el mensaje recibido al modo SOLO
    # Puedes añadir más 'elif' para otros modos aquí si los implementas.

# Pequeña pausa para ceder tiempo de CPU a otros procesos del sistema.
# Se evita si la animación de clip está en curso, ya que contiene sus propios
'time.sleep()'.
if not clipping_is_playing:
    time.sleep(0.001)

```

Resumen de funcionamiento del código

1. Configuración de Hardware

- **Vibrador (PWM):** Conectado al pin **GP29**. Utiliza modulación por ancho de pulso (PWM) para variar la intensidad de la vibración.
 - **LED NeoPixel:** Usa el LED integrado de la placa (o GP16) para indicar el estado del sistema mediante colores:
 - **Verde:** Sistema listo / Reposo.
 - **Amarillo:** Recibiendo datos / Vibración activa.
 - **Azul:** Modo "Solo" activo.
 - **Rojo:** Error o "Clipping" (saturación).
-

2. Los Dos Modos de Funcionamiento

El código cambia de comportamiento según los mensajes MIDI recibidos:

A. Modo Control MIDI (Predeterminado)

Mapea el volumen directamente a la vibración.

- Recibe mensajes de **ChannelPressure** (Presión de Canal) en el Canal 0.
- **Niveles 1-11:** El vibrador vibra proporcionalmente (1 es suave, 11 es máximo).
- **Niveles 12-13:** Activa una **animación de clip** (el vibrador y el LED rojo parpadean frenéticamente) para avisar que el audio está saturando.

B. Modo Solo

Pensado para monitorear solo las partes más fuertes de una señal.

- **Niveles 0-5:** No hay vibración (silencio táctil).
 - **Niveles 6-11:** Se re-mapean para que el nivel 6 sea el mínimo y el 11 el máximo.
 - Se activa/desactiva enviando una nota MIDI específica (**SOLO_TRIGGER_NOTE**).
-

3. Funciones Especiales y Seguridad

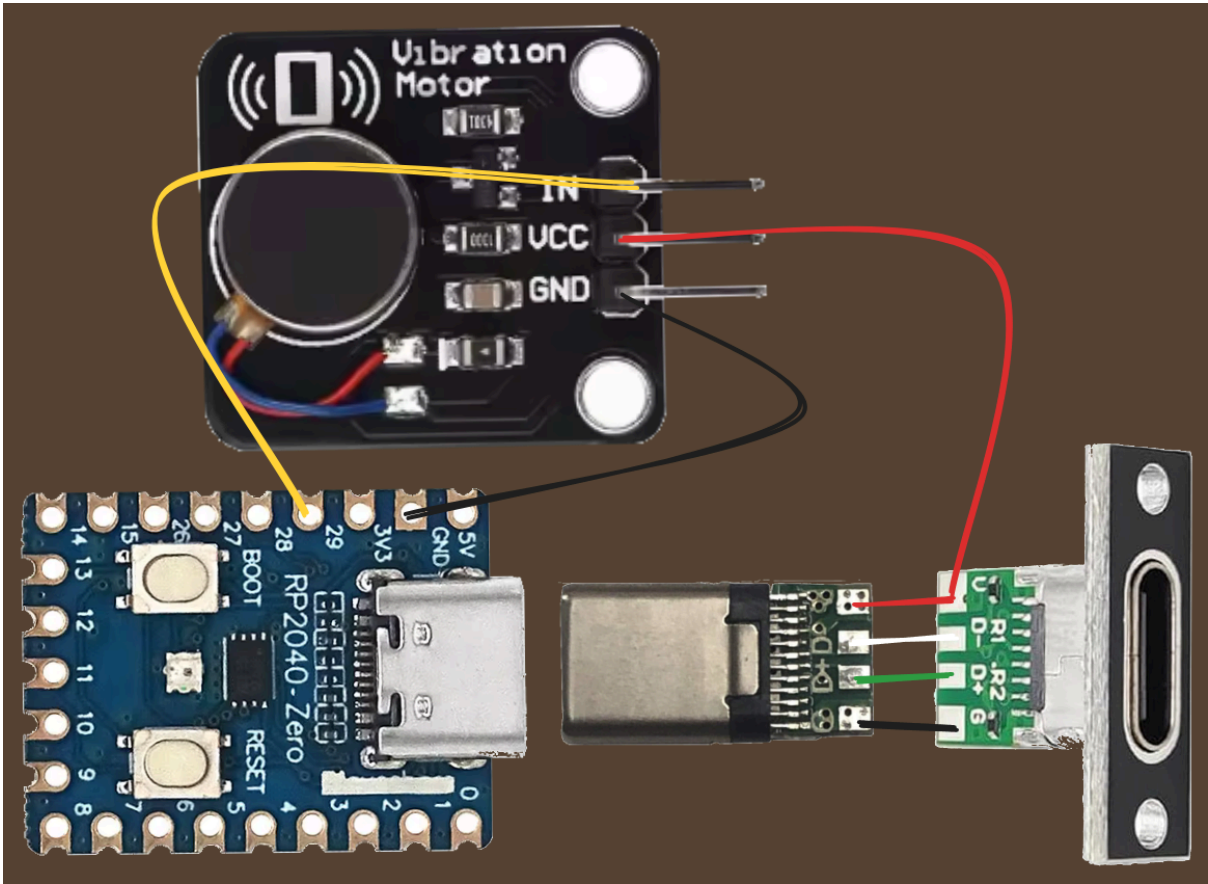
- **Animación de Inicio:** Al encenderse, el dispositivo vibra fuertemente y hace un *fade-out* (desvanecimiento) para confirmar que el motor y el LED funcionan.
 - **Sistema de Cooldown (Enfriamiento):** La animación de "clip" tiene un tiempo de espera (1 segundo) para evitar que el motor se sature o se bloquee si la señal de audio "cli pea" constantemente.
 - **Timeout de Seguridad:** Si dejan de llegar mensajes MIDI durante más de **0.5 segundos**, el vibrador se apaga automáticamente. Esto evita que el motor se quede encendido por error si el software emisor se cuelga.
-

4. Flujo del Programa (Bucle Principal)

1. **Escucha:** Revisa constantemente si hay nuevos mensajes MIDI por USB.
2. **Cambio de Modo:** Si llega una nota específica, cambia entre el modo Normal y el modo Solo.
3. **Procesa:** Según el modo activo, calcula la intensidad del PWM y actualiza el color del LED.
4. **Descansa:** Hace una pausa mínima de un milisegundo para no sobrecalentar el procesador.

En resumen: Es un controlador de **feedback háptico** que permite a un músico o técnico "sentir" los niveles de volumen y los picos de saturación de una pista de audio a través de una placa RP2040.

Esquema de conexionado



Conexión	Color del Cable
Líneas de Alimentación (+5V)	Rojo
Líneas de Tierra (GND)	Negro
Señales de Datos USB (D+/D-)	Verde y Blanco
Señal de Control PWM (GP29)	Amarillo

Documentación:

<https://docs.circuitpython.org/en/latest/shared-bindings/storage/index.html>

https://docs.circuitpython.org/en/latest/shared-bindings/usb_midi/index.html